BACHARELADO EM SISTEMAS DE INFORMAÃ§Ã£O (BSI)

Disciplina: Desenvolvimento Mobile HÃbrido

Professor ResponsÃ¡vel: **Mark Joselli** | Peso: **50% (Atividade PrÃ¡tica Somativa)**

CatÃ¡logo Interativo 2.0

Tema Escolhido: PokÃ©API (CatÃ¡logo de PokÃ©mon)

ImplementaÃ§Ã£o Integral dos Requisitos Funcionais (RF01 a RF10) com React Native & Expo SDK 51

INTEGRANTES DO GRUPO

1. Adrian AntÃ´nio de Souza Gomes	Bacharelado em Sistemas de InformaÃ§Ã£o
2. Vinicius Lima Teider	Bacharelado em Sistemas de InformaÃ§Ã£o

Data de Entrega: 02 de Novembro de 2026

SEÃ§Ã£O 1 â€” LINKS DO PROJETO

ACESSO PÃºBLICO

RepositÃ³rio GitHub:	**https://github.com/adrian69-droid/catalogo-app** (RepositÃ³rio pÃºblico com cÃ³digo-fonte completo, estrutura modular e README.md detalhando o comando `npx expo start`)
VÃdeo de ApresentaÃ§Ã£o (YouTube):	**https://youtu.be/khCRxW2I8Q4** (VÃdeo pÃºblico de 6 a 9 min com apresentaÃ§Ã£o de todos os integrantes, demonstraÃ§Ã£o funcional e explicaÃ§Ã£o tÃ©cnica)

SEÃ§Ã£O 2 â€” TEMA E CONFIGURAÃ§Ã£O

ARQUITETURA

Tema Escolhido:	**CatÃ¡logo de PokÃ©mon (PokÃ©API)** â€” Lista paginada de criaturas, busca em tempo real, visualizaÃ§Ã£o de atributos detalhados e registro de captura.
API PÃºblica Utilizada:	**PokÃ©API v2 (REST API)** â€” `https://pokeapi.co/api/v2/` (Endpoints: `/pokemon` e `/pokemon-species`). NÃ£o reutilizada do Somativo de Flutter.
OpÃ§Ã£o de PersistÃªncia:	**Local (Baseline ObrigatÃ³rio)** â€” Implementada através do `@react-native-async-storage/async-storage` (versÃ£o 1.23.1).
OpÃ§Ã£o de Login:	**Local Seguro (Baseline ObrigatÃ³rio)** â€” Cadastro e login local com sessÃ£o persistente e **hashing criptográfico de senhas (SHA-256)**.

SEÃ§Ã£O 3 â€” AUTOAVALIAÃ§Ã£O DOS REQUISITOS FUNCIONAIS

100% CONCLUÃDO

RF	DESCRIÃ§Ã£O DO REQUISITO & FUNCIONALIDADES IMPLEMENTADAS	STATUS	ARQUIVO PRINCIPAL
RF01	**CatÃ¡logo Principal:** Grade FlatList (numColumns=2), rolagem infinita (onEndReached), pull-to-refresh, componente ItemCard e placeholders de imagem.	Sim	`src/screens/CatalogScreen.js` `src/components/ItemCard.js`
RF02	**NavegaÃ§Ã£o:** Stack Navigator nativo, transiÃ§Ã£o para Detalhes passando ID por `route.params` e tÃtulo do header dinÃ¢mico via `navigation.setOptions`.	Sim	`App.js` `src/screens/DetailsScreen.js`
RF03	**Detalhes do Item:** 2Âª requisiÃ§Ã£o Axios pelo ID, imagem ampliada, badges por tipo com cores temÃ¡ticas, atributos fÃsicos, habilidades e descriÃ§Ã£o tratada (sem HTML).	Sim	`src/screens/DetailsScreen.js`
RF04	**Estado Global (Favoritos):** Context API (createContext + FavoritesProvider + hook `useFavorites()`), botÃ£o de coraÃ§Ã£o nas duas telas com sincronizaÃ§Ã£o instantÃ¢nea.	Sim	`src/contexts/FavoritesContext.js` `src/hooks/useFavorites.js`
RF05	**Tela de Favoritos:** AcessÃvel pelo catÃ¡logo, sincronizada globalmente, remoÃ§Ã£o em tempo real e estado vazio amigÃ¡vel via `ListEmptyComponent`.	Sim	`src/screens/FavoritesScreen.js`
RF06	**PersistÃªncia de Dados:** AsyncStorage para favoritos, consumidos e usuÃ¡rios. Carregamento em useEffect com indicador de loading, sem piscar dados zerados.	Sim	`src/contexts/FavoritesContext.js` `src/contexts/AuthContext.js`
RF07	**Login, SessÃ£o e Consumidos:** NavegaÃ§Ã£o condicional (Auth/App Stack), persistÃªncia de sessÃ£o, logout, senha com hash SHA-256 e tela de capturados com nota 1-5.	Sim	`src/screens/LoginScreen.js` `src/screens/ConsumedScreen.js`
RF08	**Busca:** TextInput controlado, botÃ£o "Buscar", suporte a busca por nome e #ID (ex: #001, #25), lista de resultados com FlatList e tela vazia clara.	Sim	`src/screens/CatalogScreen.js` `src/screens/SearchScreen.js`
RF09	**Feedback de UI:** ActivityIndicator em carga inicial, paginaÃ§Ã£o, busca e detalhes. Tratamento de erros com mensagem amigÃ¡vel e botÃ£o "Tentar novamente". Hook `useFetch`.	Sim	`src/hooks/useFetch.js` `src/screens/CatalogScreen.js`
RF10	**Acessibilidade:** accessible, accessibilityLabel, accessibilityRole, accessibilityState, hitSlop mÃnimo de toque (44x44pt), allowFontScaling ativo e alto contraste.	Sim	`Todos os componentes e telas`

SEÃ§Ã£O 4 â€” COMPARATIVO TÃ©CNICO: FLUTTER Ã— REACT NATIVE

ANÃ¡LISE COMPARATIVA

1. Estado Global

PROVIDER Ã— CONTEXT

No Flutter, utilizamos o pacote `provider` com classes herdadas de `ChangeNotifier`, consumidas através de `Consumer` ou `context.watch()`. Embora eficiente, exige criação de classes dedicadas e despacho manual de `notifyListeners()`. No React Native, a **Context API** nativa combinada com hooks customizados (`useFavorites()`, `useAuth()`) integrou-se com muito menos boilerplate ao ciclo do React. No entanto, o Context API exige atenção no particionamento dos Providers para evitar re-renderizações em cascata em árvores profundas.

2. Listas e Grade

GRIDVIEW × FLATLIST

No Flutter, a exibição em grade utilizou `GridView.builder` com `SliverGridDelegateWithFixedCrossAxisCount` e monitoramento de paginação via `ScrollController`. Já no React Native, o `FlatList` simplificou expressivamente o catálogo através da prop `numColumns={2}`, gerenciando a virtualização de memória nativa do viewport, suporte integrado a pull-to-refresh (`onRefresh`) e anexação incremental via `onEndReached` e `onEndReachedThreshold`.

3. Assincronismo

FUTUREBUILDER × USEEFFECT / USESTATE

No Flutter, o consumo de APIs baseia-se amplamente no widget `FutureBuilder` e checagem de `AsyncSnapshot` (ConnectionState.waiting, hasError). No React Native, adotamos a composição funcional com `useEffect` e estados explícitos de `loading`, `error` e `data` (encapsulados em nosso hook `useFetch`). Essa abordagem do React Native revelou-se mais flexível para acumular dados paginados sem desmontar ou reiniciar o estado da tela como por vezes ocorre no FutureBuilder.

4. Ferramentas

HOT RELOAD × FAST REFRESH / EXPO

Enquanto o Hot Reload do Flutter é veloz para ajustes visuais puramente declarativos, o **Fast Refresh do React Native** destacou-se pela robustez ao preservar o estado interno de formulários e hooks entre salvamentos. Além disso, o ecossistema **Expo Go** facilitou drasticamente o fluxo de trabalho do grupo, permitindo testes instantâneos em múltiplos smartphones físicos (Android e iOS) via QR Code, sem necessidade de builds demorados no Gradle ou Xcode.

SEÃ§Ã£O 5 â€” TEMA DA EXPLICAÃ§Ã£O TÃ©CNICA NO VÃDEO

APROFUNDAMENTO

Tema Escolhido: Context API vs. Estado Local e NavegaÃ§Ã£o Condicional

Minutagem no VÃdeo: 05:30 Ã s 08:00

O grupo aborda em profundidade a tomada de decisão arquitetural entre o isolamento de estados locais (`useState`) e a centralização de estados globais compartilhados via **Context API**. Destacamos por que entidades como autenticação/sessão (`AuthContext`) e lista de favoritos/capturados (`FavoritesContext`) precisam residir no topo da árvore de componentes, evitando o problema de *prop drilling* entre telas irmãs e garantindo que o toque no coração de um card no catálogo reflita instantaneamente na tela de detalhes e favoritos.

Adicionalmente, explicamos o mecanismo de **navegação condicional** implementado no arquivo `App.js`: como o estado `signed` do `useAuth()` alterna dinamicamente entre o stack de autenticação (`LoginScreen`) e o stack autenticado do aplicativo, elucidando tecnicamente por que essa abordagem é superior ao uso de `navigation.navigate('Login')` após um logout, eliminando o histórico de rotas anteriores e impedindo acessos indevidos via botão "Voltar" do sistema operacional.

SEÃ§Ã£O 6 Â€” TELAS PRINCIPAIS DO APLICATIVO

DEMONSTRAÃ§Ã£O VISUAL

1. AUTENTICAÃ§Ã£O & SESSÃ£O

ðŸ”

Tela de Login e Cadastro

Abas alternÃ¡veis, senha protegida com hash SHA-256 e credenciais prÃ©-carregadas.

LoginScreen.js

NavegaÃ§Ã£o condicional â€¢ SessÃ£o persistida

2. CATÃ¡LOGO EM GRADE

âš¡

Grade de PokÃ©mon (2 Colunas)

Rolagem infinita com paginaÃ§Ã£o, pull-to-refresh e chips de acesso rÃ¡pido.

CatalogScreen.js

FlatList â€¢ numColumns={2} â€¢ ItemCard

3. DETALHES & AVALIAÃ§Ã£O

ðŸ“Š

Ficha TÃ©cnica do PokÃ©mon

Badges por tipo elemental, peso, altura, botÃ£o de favorito e avaliaÃ§Ã£o 1 a 5 estrelas.

DetailsScreen.js

Axios por ID â€¢ Header dinÃ¢mico â€¢ Rating

4. BUSCA INTELIGENTE

ðŸ”

Resultados por Nome ou #ID

Busca por nome (ex: Pikachu) ou nÃºmero (#001, #25, 25) com lista e fallback.

SearchScreen.js

Input controlado â€¢ SanitizaÃ§Ã£o de # e zeros

5. MEUS FAVORITOS

â¤ï¸

Lista Global de Favoritos

SincronizaÃ§Ã£o reativa instantÃ¢nea e ListEmptyComponent amigÃ¡vel.

FavoritesScreen.js

Context API â€¢ useFavorites() â€¢ AsyncStorage

6. ITENS CAPTURADOS

ðŸŽ’

Itens Consumidos

Listagem com classificaÃ§Ã£o pessoal de 1 a 5 estrelas douradas e persistÃªncia.

ConsumedScreen.js

AvaliaÃ§Ã£o pessoal â€¢ PersistÃªncia local